Density, Probability and Order in Local Code Retrieval: Calibrated Tracing, Recursive Context Quality and Cache-Blocked Execution

First draft — not for circulation

Moksh Shah

September 2026

Abstract

We study retrieval-augmented question answering and editing over a single source repository with a small language model that runs entirely on a laptop. Four questions organise the work. How should functions that are semantically alike be grouped when their embeddings are unit vectors in a space of hundreds of dimensions? When should retrieval stop, so that everything the model needs is present and little else is? How can the quality of an assembled context be measured, so that "context rot" becomes a quantity rather than an impression? And what can user-space software actually guarantee about the order and the cache residency of parallel work? We answer with (i) directional kernel density estimation with the von Mises–Fisher kernel and mode seeking by directional mean shift, with a concentration limited by a neighbourhood criterion after we observe that likelihood cross-validation fails for mode seeking above a few dimensions; (ii) a calibrated Bayes posterior of relevance, propagated along call-graph and density edges by a noisy-OR rule and cut at a threshold τ whose value is a statement about costs, $\tau = c_{\mathrm{fp}}/(c_{\mathrm{fp}} + c_{\mathrm{fn}})$; (iii) a recursive context-quality measure with a pruning rule that guarantees a lower bound on quality and whose iteration converges to a unique fixed point; and (iv) a block-size rule derived from the cache hierarchy together with an ordered executor that provably admits and commits work in sequence order, making every numerical result independent of the number of threads. All measurements are taken on a single corpus, the *Demo repository*, which is a fork of the mimalloc memory allocator [46]. On this corpus the groups recover the call graph (35.8% of call edges fall inside a group against 4.9% at random); the thresholded posterior returns sets whose precision matches τ but, with an embedder that ranks the relevant function first for 23% of questions, little recall; the context-quality index correlates with the loss a context causes (Spearman $\rho = 0.36$); execution is bit-identical across 6 168 runs at every thread count and block size; and the promotion test rejects a fine-tuned model that improved on the corpus while forgetting general text.

1 Introduction

Long prompts hurt language models even when the relevant text is present. Accuracy falls as input length grows [31], it falls with length alone even when retrieval is perfect [16], and it depends on where in the prompt the evidence sits [43]. Operational measures of the effect exist, such as the effective length of NoLiMa, the largest length at which a model keeps 85% of its short-context score [47], but no scalar measure of the *quality* of a given context has become standard. For code the problem has a structure that text lacks: a function is a natural unit, functions call each other, and removing single tokens of code corrupts it [53, 61].

This paper treats retrieval over one repository as a sequence of statistical decisions made under explicit assumptions. Section 2 fixes notation and the corpus. Section 3 derives the size of the unit of parallel work from the cache hierarchy and proves the ordering guarantees of the executor that runs it. Section 4 groups functions by the modes of a directional kernel density. Section 5 turns similarity scores into calibrated probabilities and extends retrieval along the call graph until the probability of relevance falls below τ. Section 6 defines recursive context quality and the pruning rule. Section 7 bounds the error introduced by compressed vector codes. Section 8 describes opportunistic fine-tuning and quantisation of the local model and the test that decides whether a tuned model replaces the served one. Section 9 gives the acceptance criterion for applying model-proposed edits. Section 10 reports the experiments.

Contributions. (1) A block-size rule and an ordered executor with admission and commit order guarantees and bit-identical reductions (Theorem 1). (2) Evidence that likelihood cross-validated concentration is unsuitable for directional mean shift above a few dimensions, and a neighbourhood limit that restores planted group-

ings (Section 4). (3) A calibrated probabilistic trace with a closed-form reach condition (Proposition 3). (4) Recursive context quality with an order-independent threshold guarantee and a unique, greedy fixed point (Theorems 7 and 6). (5) A promotion test for locally fine-tuned models. (6) An evaluation on one repository that includes the negative results: low recall of the thresholded trace with a weak embedder, no measurable benefit of graph edges, catastrophic forgetting caught by the promotion test, and a defect in edit scoping found and fixed by the evaluation. Items (3)–(5) combine known ingredients; what is new is their combination into one decision procedure and the proofs that tie them together. Several ingredients have close prior art, which we cite where they appear.

2 Setting

Corpus. All experiments use the Demo repository, a fork of mimalloc [46] at commit `31d034d9` (version 3.5.3). Its C sources and headers under `src/` and `include/` comprise 45 files and 1 090 189 bytes. The results therefore describe one C code base with heavy use of macros and conditional compilation, and may not transfer to other languages or repositories.

Units. A file is partitioned into contiguous byte ranges called *units*: function definitions with their leading comment, type definitions, macros, and residual regions. Parsing combines a concrete syntax tree [55] with brace matching over a token stream, because the declaration macros of the corpus defeat the grammar in 37 of the 45 files. The corpus yields $n = 2326$ units: 1514 functions, 70 types, 341 macros and 401 regions. Every definition found by an independent brace-matching count (1427 bodies) is recovered. The *call graph* $G = (V, E)$ has an edge $u \to v$ when the body of u calls a name defined by v; it has 5800 edges.

Embeddings. Each unit v has an embedding $e_v \in S^{D-1}$, the unit sphere in $\mathbb{R}^D$; the question has an embedding $q \in S^{D-1}$, and the score of v is $s_v = \langle q, e_v \rangle$. We use Qwen3-Embedding-0.6B [59] with $D = 1024$, last-token pooling and an instruction prefix on questions, run locally. A model-free signed feature-hashing embedder serves as a baseline and for tests.

Machine. The reference machine has an Intel Core i9-12950HX: 8 performance cores with a 48 KiB L1 data cache shared by two hyper-threads, 8 efficiency cores with a 32 KiB L1 data cache each, 24 logical CPUs, 32 GB of memory, and no GPU in use.

3 Cache-blocked ordered execution

3.1 Block size

Software cannot choose the cache level that holds its data. It can choose the size of its working set, and a working set that fits the first-level data cache stays there after its first use, apart from interference by other code [22, 39]. Processing data in cache-sized vectors is a long-standing technique in query execution [9, 40]. Let the machine have core classes c, each with an L1 data cache of $C(c)$ bytes shared by $h(c)$ logical CPUs. We choose the block size

$$B = \left\lfloor \min\Big(f \cdot \min_c C(c),\ \min_c \tfrac{C(c)}{h(c)} \Big) \right\rfloor_{\mathrm{KiB}}, \qquad f = \tfrac{1}{2}. \quad (1)$$

The first term leaves half of the smallest L1 data cache for the query, the outputs and the stack, following the recommendation to size blocking to a fraction of the first-level cache [35]. The second term keeps the block within the share of one logical CPU when its hyper-thread sibling is busy. On the reference machine $\min(32/2,\ \min(48/2, 32/1)) = 16$ KiB.

A block pays off only if it is touched more than once. A single streaming pass over the data gains nothing from being L1-sized, so every pairwise computation in this work is *tiled*: a data block is kept hot while all queries or seeds of a batch pass over it.

3.2 Ordered execution

Blocks carry sequence numbers $0, 1, \ldots, N-1$. Workers obtain blocks from a shared counter (dynamic assignment, which adapts to the unequal speeds of the two core classes [34]). A block goes through three events: *admission* (the first read of its payload, performed inside a gate), *work* (concurrent), and *commit* (sequential). Two counters a (admitted) and k (committed) and a reorder window W define the protocol: worker holding ticket i may admit when $a = i$ and $i < k + W$, then sets $a \leftarrow i + 1$; after the work it marks i done and, under a mutex, commits the longest run of done blocks starting at k, advancing k each time. The gate is a FIFO ticket gate [45].

Theorem 1 (Order)**.** *For every execution: (G1) the admission of block i precedes the admission of block $i+1$; (G2) commit i completes before commit $i+1$ starts, and no two commits overlap; (G3) if no work item fails, each block is admitted, worked and committed exactly once; (G4) block i is not admitted before block $i - W$ has been committed.*

Proof. (G1) a takes the values $0, 1, 2, \ldots$ in order, since only the holder of ticket a may pass the gate and it increments a after its admission. Block $i + 1$ passes only

when $a = i + 1$, which happens after block i was admitted. (G2) Commits happen under one mutex, and the drain commits block k and then increments k; block $i+1$ can be committed only when $k = i+1$. (G3) Tickets are distinct values of an atomic counter, so each block has exactly one owner; the drain visits each k once. (G4) The gate condition $i < k + W$ implies $k > i - W$ at admission. □

Corollary 2 (Determinism)**.** *If every reduction is performed in the commit step, its result is a function of the inputs alone: floating-point sums and top-k merges are bit-identical for every thread count and every schedule.*

The corollary holds by construction and is tested: a sum of 200 000 single-precision values with a heavy-tailed distribution is bit-identical for 1, 2, 5, 16 and 24 threads over repeated runs, and a logical clock records zero violations of G1 and G2 in all tests. What the theorem does not state is when a block's work runs relative to another's; the operating system may pre-empt a worker between its admission and its work, and determinism does not require otherwise. Deterministic multithreading systems obtain stronger guarantees for arbitrary programs [6, 44, 48]; ordered reduction needs only the commit order.

Speed-up bound. Let one block take d time units at the gate, w in work and c in commit, on P equally fast workers. Gate and commit are serial, so in steady state

$$S(P) \le \min\Big(P,\ \frac{d + w + c}{\max(d, c)}\Big), \tag{2}$$

a pipeline form of Amdahl's bound [2, 30]. With a gate that only reads a few cache lines and a commit that only writes a block's scores, $d, c \ll w$.

4 Grouping by directional density

4.1 Model

Embeddings are directions, so we estimate their density on the sphere S^{m-1} after reduction to m dimensions. The von Mises–Fisher density with mean μ and concentration κ is [5]

$$f(x; \mu, \kappa) = C_m(\kappa)\, e^{\kappa \langle \mu, x \rangle}, \tag{3}$$

$$C_m(\kappa) = \frac{\kappa^{m/2-1}}{(2\pi)^{m/2}\, I_{m/2-1}(\kappa)}, \tag{4}$$

with I_ν the modified Bessel function of the first kind. The kernel density estimator with the von Mises kernel [4, 29] is a mixture of n such densities with common concentration:

$$\hat{f}_\kappa(x) = \frac{C_m(\kappa)}{n} \sum_{i=1}^{n} e^{\kappa \langle x_i, x \rangle}. \tag{5}$$

All evaluations are done in the logarithmic domain; $\log I_\nu$ is computed from its power series, its uniform (Debye) asymptotic expansion, or its large-argument expansion depending on the regime, to a relative error below 10^{-9} in the logarithm for $\nu \le 512$ and arguments up to 10^6.

Preprocessing. Embeddings of language models are anisotropic: they occupy a narrow cone [20], and nearest neighbours in high dimension concentrate on a few hubs [51]. Both effects bias a kernel density toward the common direction. We centre the embeddings on their mean, project on the leading principal directions ($m = 32$ by default), and re-normalise to the sphere. Non-parametric estimation converges at the rate $n^{-2/(m+4)}$ [54], so reduction is necessary at $n \approx 2300$; a random projection preserving pairwise distances to 20% would need more than 1700 dimensions at this n [14], which is why a data-dependent projection is used and validated empirically rather than justified by that bound.

4.2 Mode seeking

Directional mean shift [60] iterates, from every data point,

$$x \leftarrow \frac{\sum_i w_i x_i}{\big\| \sum_i w_i x_i \big\|}, \qquad w_i = e^{\kappa(\langle x_i, x \rangle - 1)}. \tag{6}$$

It is gradient ascent on $\hat{f}_\kappa$ with an adaptive step, the density is non-decreasing along each trajectory, and under regularity conditions the iteration converges to a mode [60]; it is the directional counterpart of Euclidean mean shift [12, 13, 23]. Points whose iterations end within cosine 0.999 of each other form one group; groups smaller than three points join the nearest mode. Storing units group by group, in decreasing density, puts similar functions in adjacent memory, which the index of Section 7 exploits.

4.3 Choice of concentration

The directional rule of thumb [25] is the AMISE-optimal bandwidth when the data are themselves vMF. With $q = m-1$ and $\hat{k}$ the maximum-likelihood concentration of the whole sample,

$$h^{4+q} = \frac{4\sqrt{\pi}\, I_{\frac{q-1}{2}}(\hat{k})^2 \,/\, n}{\hat{k}^{\frac{q+1}{2}} \big[2q\, I_{\frac{q+1}{2}}(2\hat{k}) + (2+q)\, \hat{k}\, I_{\frac{q+3}{2}}(2\hat{k})\big]} \tag{7}$$

and $\kappa = h^{-2}$. We verified this expression against numerical quadrature of its definition to 5×10^{-13} in the logarithm for $q \in \{1, 2, 3, 7, 15, 31\}$. Likelihood cross-validation then maximises $\sum_i \log \hat{f}_\kappa^{-i}(x_i)$ over a grid centred on the rule of thumb.

Table 1: Directional mean shift at the cross-validated concentration on planted vMF mixtures (three seeds each). "Singletons" is the share of points forming a mode of their own; ARI is the adjusted Rand index against the planted labels.

m	n	singletons	ARI
3	500	0–0.2%	0.997–1.000
8	600	22–31%	0.44–0.84
16	400	99–100%	0.000
32	2000	74–77%	0.32–0.35

Observation: cross-validation fails for mode seeking. On planted mixtures of vMF components, the cross-validated κ gives held-out likelihood within 0.04% of the best, as intended, but it is far too large for mode seeking once m exceeds a few dimensions (Table 1). Kernels then no longer reach the neighbours of a point, every point is a local maximum of $\hat{f}_\kappa$, and mean shift returns it as its own mode.

We therefore limit the concentration. With $N_i(\kappa) = \sum_{j \neq i} e^{\kappa(\langle x_i, x_j \rangle - 1)}$, the weight of the other points at x_i in units of the weight of x_i itself, N_i decreases in κ, and a point with $N_i \ll 1$ is a local maximum on its own. We define the *neighbourhood concentration* κ_N as the largest κ for which at most 5% of the points have $N_i < 1$, and use $\min(\kappa_{\mathrm{CV}}, \kappa_N)$ for mode seeking while keeping κ_{CV} for density values. On a mixture of 12 components in $m = 32$ with $n = 2000$, $\kappa_{\mathrm{CV}} = 102.8$ and $\kappa_N = 10.7$; at κ_N mean shift finds exactly the 12 modes, ARI 0.996, in 6 steps per point on average, none unconverged. The constants of the criterion (5%, $N_i < 1$) are heuristic, and the criterion has so far been validated on synthetic mixtures only.

5 Calibrated relevance and probabilistic tracing

5.1 From scores to probabilities

Scores are standardised against the background of the question itself, $z_v = (s_v - \bar{s})/\hat{\sigma}_s$, which removes the question-dependent offset and scale. Let f_1 and f_0 be the densities of z for relevant and non-relevant units and π the prior probability of relevance. Bayes' rule gives

$$P(\mathrm{rel} \mid z) = \sigma\big(\operatorname{logit} \pi + \ln f_1(z) - \ln f_0(z)\big). \tag{8}$$

We estimate f_1, f_0 by Gaussian kernel density estimation with Silverman's bandwidth [49, 52] on labelled calibration pairs, tabulate (8) on a grid, and project the table onto non-decreasing functions by isotonic regression weighted by the mixture density $\pi f_1 + (1-\pi) f_0$. The projection matters: without monotonicity, the rule "accept while the probability is at least τ" can have several stopping points. Calibration of this kind is necessary because raw scores of neural models are not probabilities [27].

Calibration pairs come from the corpus: a function's documentation is a question to which the function is the answer. To prevent the question from matching its own text, the relevant unit is embedded without its leading comment. Reliability on held-out pairs is reported as expected calibration error and Brier score.

5.2 The threshold as a cost ratio

Admitting a unit with relevance probability p costs $c_{\mathrm{fp}}(1-p)$ in expectation (context rot); omitting it costs $c_{\mathrm{fn}} p$ (a missed dependency). Admission is optimal iff $p \geq c_{\mathrm{fp}}/(c_{\mathrm{fp}} + c_{\mathrm{fn}})$ [17]. The threshold $\tau = 0.25$ used throughout therefore states that missing a relevant function is three times as costly as admitting an irrelevant one. It is meaningful only for calibrated p, and the ratio is expected to change when fine-tuning makes the model more tolerant of distractors. A complementary, distribution-free choice is the split-conformal threshold, the $\lceil (n+1)(1-\alpha) \rceil$-th largest calibration score of relevant units, which retains a relevant unit with probability at least $1 - \alpha$ under exchangeability [3, 11].

5.3 Tracing

Relevance is correlated along the call graph: if a function is relevant, its callees often are. Let $p_s(v) = P(\mathrm{rel} \mid z_v)$ and let an edge of kind $e \in \{\mathrm{callee}, \mathrm{caller}, \mathrm{sibling}\}$ carry a likelihood ratio $\lambda_e \geq 1$, where siblings are nearest neighbours inside the same density mode. Given that an accepted neighbour u is relevant,

$$p(v \mid u) = \sigma\big(\operatorname{logit} p_s(v) + \gamma_{uv} \ln \lambda_e\big), \tag{9}$$

where $\gamma_{uv} = 1/(1 + \log_2 \max(1, \deg))$ damps edges into hubs (utility functions called from everywhere carry little information). Since u is relevant only with probability $P(u)$, the evidence u provides is the mixture

$$q_u(v) = P(u)\, p(v \mid u) + (1 - P(u))\, p_s(v) \;\geq\; p_s(v), \tag{10}$$

and evidence from several accepted neighbours is combined by a noisy-OR of the normalised gains $d_u(v) = (q_u(v) - p_s(v))/(1 - p_s(v)) \in [0, 1]$:

$$P(v) = 1 - (1 - p_s(v)) \prod_u \big(1 - d_u(v)\big). \tag{11}$$

Units with $p_s \geq \tau$ are seeds. Candidates are then expanded best-first; a unit is accepted when $P(v) \geq \tau$, and its probability is frozen at acceptance, which excludes feedback between mutually adjacent units. Tracing stops when no candidate reaches τ. Personalised PageRank propagates relevance on a graph in a related way [18, 28]; its stationary vector sums to one over all units, so a probability threshold cannot be applied to it directly, whereas (11) is a probability per unit.

Proposition 3 (Reach). *A unit reached through a single edge of kind e from a neighbour accepted with $P(u) \to 1$ is accepted iff*

$$p_s(v) \;\geq\; \frac{\tau}{\tau + \lambda_e^{\gamma}(1-\tau)}. \tag{12}$$

Proof. With $P(u) = 1$, $P(v) = p(v \mid u) = \sigma(\operatorname{logit} p_s(v) + \gamma \ln \lambda_e) \geq \tau$ iff $\operatorname{logit} p_s(v) \geq \operatorname{logit} \tau - \gamma \ln \lambda_e$, i.e. $\frac{p_s}{1-p_s} \geq \frac{\tau}{(1-\tau)\lambda_e^{\gamma}}$, which rearranges to the claim. □

For $\tau = 0.25$ and an undamped callee edge with $\lambda = 4$ the bound is $1/13 \approx 0.077$. The proposition also bounds the work of the engine: only units whose score could reach this posterior need exact rescoring when compressed codes are used (Section 7).

Proposition 4 (Well-posedness). *$P(v) \geq p_s(v)$ for every unit; each unit is accepted at most once, so tracing terminates after at most n acceptances; and the set of seeds is monotone in τ.*

Proof. $\lambda_e \geq 1$ gives $p(v \mid u) \geq p_s(v)$, hence $q_u \geq p_s$, $d_u \in [0,1]$ and $P(v) \geq p_s(v)$ by (11). Acceptance is irreversible, and $\{v : p_s(v) \geq \tau\}$ shrinks as τ grows. □

Randomised tests on 200 graphs additionally find the accepted sets nested across $\tau \in \{0.1, 0.25, 0.5\}$; nesting of the traced (not only the seed) sets is not proved here. Noisy-OR treats the evidence of different neighbours as independent; on graphs whose paths share edges it overstates the combined evidence, and the lifts λ_e are therefore estimated from data (Section 10).

6 Recursive context quality

6.1 Definition

A context is a tree: context → files → units. For a unit u with calibrated relevance p_u and length $L(u)$ tokens let $M(u) = p_u L(u)$. For an inner node v with children $u_1, \ldots, u_k$ in prompt order and overhead $h(v)$ tokens,

$$L(v) = h(v) + \textstyle\sum_j L(u_j), \tag{13}$$

$$M(v) = \textstyle\sum_j \big(1 - \rho(u_j)\big) M(u_j), \tag{14}$$

$$\rho(u_j) = \max_{i<j} \max\big(0, \cos(e_{u_i}, e_{u_j})\big), \tag{15}$$

and the quality is $Q(v) = M(v)/L(v) \in [0,1]$. The *context rot index* of a context C is $\mathrm{CRI}(C) = 1 - Q(C)$: the expected share of prompt tokens that are irrelevant, redundant or overhead. Only units carry embeddings, so in practice redundancy is taken against every unit placed earlier in the whole prompt and $Q(C) = \sum_u (1-\rho_u) p_u L_u / L(C)$. The measure generalises the flat context-relevance ratio of RAGAs [19] by weighting by probability, discounting redundancy inside the recursion and charging structural overhead. The redundancy term is that of MMR [10], which, unlike monotone submodular objectives, carries no approximation guarantee [41]; we use it only inside a threshold rule, for which Theorem 7 gives a guarantee directly.

Lemma 5. *Removing child u from v, other redundancies fixed, raises $Q(v)$ iff $(1-\rho(u))\,Q(u) < Q(v)$.*

Proof. $Q(v \setminus u) = \frac{M(v)-(1-\rho_u)M(u)}{L(v)-L(u)} > \frac{M(v)}{L(v)}$ iff $(1-\rho_u)M(u)L(v) < M(v)L(u)$, i.e. $(1-\rho_u)Q(u) < Q(v)$. □

Unconstrained maximisation of Q would therefore keep only the single best unit; a threshold with a recall safeguard is needed, which is what the pruning rule provides.

6.2 Pruning

Sort candidates by p (descending, ties by identifier) and keep u iff $(1-\rho(u))p_u \geq \theta$, with ρ taken against the *kept* candidates ranked before u. Because dropping a unit can make a later one non-redundant, the rule is iterated: each pass recomputes ρ against the units kept by the previous pass.

Theorem 6 (Fixed point). *The iteration reaches a unique fixed point within n passes, and the fixed point equals the greedy selection that scans candidates once in rank order.*

Proof. The status of the j-th candidate depends only on the statuses of candidates $1, \ldots, j-1$. The first candidate's status is final after one pass; if the first j statuses are final after j passes, the $(j+1)$-th is final after pass $j+1$. The greedy scan computes exactly these statuses in order. □

The kept set is not monotone across passes (a unit can return), so a termination argument by shrinking sets does not apply; the argument above does.

Theorem 7 (Quality bound). *At the fixed point every kept pair i, j satisfies $(1-c_{ij})\min(p_i, p_j) \geq \theta$ with $c_{ij} = \max(0, \cos(e_i, e_j))$. Hence every kept unit has discounted quality at least θ against* any *subset of the other kept units, whatever the prompt order, and*

$$Q(C) \;\geq\; \theta\, \frac{L_{\text{kept}}}{L_{\text{kept}} + L_{\text{support}} + L_{\text{overhead}}}. \tag{16}$$

Proof. If j is kept and i was kept before it, $(1-c_{ij})p_j \geq \theta$ and $p_i \geq p_j$. Redundancy against a subset is a maximum over fewer terms, hence no larger. Summing $(1-\rho_u)p_u L_u \geq \theta L_u$ over kept units and dividing by $L(C)$ gives the bound. □

Support units are skeletons (signature and documentation) of callees that a kept unit needs, added after pruning; enforcing such dependency closure matters for code [56]. The utility of [56], $U(S) = \sum_{i \in S}(r_i - \lambda c_i) -$

$M\,\big|\{(i,j) \in E : i \in S,\ j \notin S\}\big|$, is claimed to have only dependency-closed optima when $M > \max_j (r_j - \lambda c_j)_+$. This is false as stated: for the chain $a \to b \to c$ with net values $100, 100, -1000$ and $M = 101$, the open set $\{a, b\}$ scores 99 while every closed set scores at most 0 (checked by enumeration). The argument fails because adding the missing node can itself be costly; the claim does hold when every net value is non-negative. We therefore enforce closure as a hard constraint rather than through a penalty. When the result exceeds the token budget, units outside a protected share of the budget are first reduced to skeletons and then dropped in increasing order of discounted mass; removal only lowers redundancy, so the bound survives. Finally, files are ordered strongest first, second strongest last, following the positional effect of [43]; for question answering over retrieved documents, noise has been found to cost several times more accuracy than position, so pruning precedes ordering.

7 Compressed scoring with error bounds

The scan of Section 3 scores compressed codes. With RaBitQ [24], vectors are centred on the corpus centroid, normalised to o, rotated by a random orthogonal P, and reduced to signs, giving $\bar{o} = P\,\mathrm{sign}(P^\top o)/\sqrt{D}$. For a centred, normalised query q', the estimator $\langle \bar{o}, q' \rangle / \langle \bar{o}, o \rangle$ is unbiased for $\langle o, q' \rangle$, and with high probability its error is below

$$\sqrt{\frac{1 - \langle \bar{o}, o \rangle^2}{\langle \bar{o}, o \rangle^2}} \cdot \frac{\epsilon_0}{\sqrt{D-1}}. \tag{17}$$

With $D = 1024$, the typical $\langle \bar{o}, o \rangle \approx 0.8$ and $\epsilon_0 = 1.9$ the half-width is $0.75 \times 1.9/\sqrt{1023} \approx 0.045$. Each estimate is therefore an interval, and a unit is rescored exactly when its interval contains the score at which its posterior can still reach the bound of Proposition 3; decisions then rest on exact scores only. A code occupies $D/8 = 128$ bytes plus two floats, so a 16 KiB block holds about 120 vectors. For the alternative symmetric int8 code with scale s, each coordinate carries a rounding error of at most $\delta = s/254$; for unit vectors and no clipping the inner-product error is at most $2\delta\sqrt{D} + \delta^2 D$.

8 Opportunistic fine-tuning and quantisation

The served model is Qwen2.5-Coder-0.5B-Instruct [33] quantised to 4.5 bits per weight (Q4_K_M [36]). When the user enables tuning and the machine is idle, the system builds training examples from the repository in the style of RAFT [58]: a question, the unit that answers it, and distractors drawn from the same density mode and from units whose posterior is closest to τ, with the answering unit absent from a fixed share of examples so the model learns to say so. Units are split into training and held-out sets before examples are built.

Training. Low-rank adaptation [15, 32] is the usual choice, but the local training API (llama.cpp) trains float32 weights directly and has no low-rank trainer upstream [26]. We therefore update only the last b transformer blocks: $\theta = (\theta_{\text{frozen}}, \theta_{\text{top}})$ with $\nabla_{\theta_{\text{frozen}}} = 0$, which bounds memory and the drift from the base model, a known risk of fine-tuning [7, 38]. Training runs only while the machine is idle and is cancelled by any user activity, in the spirit of eligibility rules for on-device learning [8].

Quantisation. The tuned model is quantised with an importance matrix $I_j = \mathbb{E}[x_j^2]$, the mean squared activation of input column j of each weight matrix over repository text in the serving prompt format [37]. The motivation is the diagonal approximation

$$\mathbb{E}\big\|(W - \hat{W})x\big\|^2 \approx \sum_{r,j} I_j\,(W_{rj} - \hat{W}_{rj})^2, \tag{18}$$

related to the second-order objectives of GPTQ [21] and activation-aware scaling [42]; the implemented k-quant objective weights errors by a function of I_j and the weight magnitude rather than by I_j alone.

Promotion test. A candidate replaces the served model only if (i) its held-out domain loss is lower in a one-sided paired t-test at level α over held-out examples, with $d_i = \ell_i^{\text{base}} - \ell_i^{\text{cand}}$ and $t = \bar{d}/(s_d/\sqrt{n})$; (ii) its loss on general text is not worse by more than a margin δ (non-inferiority); and (iii) the mean per-token Kullback–Leibler divergence between the quantised and unquantised candidate stays below a bound. The previous model is kept for rollback.

9 Applying proposed edits

The model proposes edits as *search/replace* blocks or as the complete new text of one named unit; line numbers are never used [1, 57]. A search text s is located by a ladder of comparisons of decreasing strictness: exact bytes; ignoring trailing blanks; ignoring leading and trailing blanks (the replacement is then re-indented); and, only when enabled, a fuzzy rung.

Definition 1 (Acceptance)**.** A location is accepted iff it is the *only* match on the first rung that matches anything, inside the named unit when one is given. On the fuzzy rung a window of lines with edit distance d to s is accepted iff $d \le \lfloor 0.1|s| \rfloor$ and no other non-overlapping window is within that bound.

Ambiguity is refused rather than resolved, so the only way to apply an edit at the wrong place is for the intended text to be absent and an unintended text to match uniquely. All blocks for one file are resolved against one snapshot, must not overlap, and are applied as one transaction: a journal and backups are written first, the files are replaced by write-then-rename, and removing the journal is the commit point [50]. Edits that increase the number of syntax errors reported by the parser are refused.

10 Experiments

All experiments ran on the reference machine against the Demo repository, with Qwen3-Embedding-0.6B (8-bit) for embeddings and Qwen2.5-Coder-0.5B-Instruct (Q4_K_M) as the served model, in one run of five hours. Every number below is read from the result files of that run; 95% confidence intervals are bootstrap intervals over queries unless stated otherwise.

Query sets. Two question sets were built from the corpus. *Q-doc* uses the documentation of a function (at least six words) as the question and the function as the single relevant unit; the unit is embedded without its documentation so that the question cannot match its own text. Only 271 functions qualify; they were split by function into 142 calibration, 72 held-out and 57 test questions. *Q-gen* asks the served model to write a question about a function without naming it. The 0.5B model named the function in most of its attempts, and only 51 questions survived the filter (36 calibration, 9 held-out, 6 test). Q-gen results are therefore reported only where noted, and with that caveat.

10.1 Grouping (E3)

The raw embeddings are strongly anisotropic: the mean pairwise cosine is 0.445. Centring removes this (-2×10^{-5}), and the skewness of the 10-occurrence distribution falls from 1.10 to 0.86; reduction to $m = 32$ dimensions keeps 62% of the variance.

On the corpus the failure of cross-validated concentration seen on synthetic data recurs. In $m = 32$ the cross-validated value is $\kappa_{\mathrm{CV}} = 188.5$ and the neighbourhood limit $\kappa_N = 10.8$. Table 2 shows the number of modes as κ grows: one mode up to $\kappa_N/8$, 50 modes with two singletons at κ_N, and from $2\kappa_N$ on mostly singletons (608 modes, 380 of them single points); at $\kappa_{\mathrm{CV}} \approx 17\kappa_N$ nearly every unit would be its own group. With κ_N and groups of at least three units, mean shift finds 44 groups in 24 steps per point on average, every point converges, and the density never decreases along a trajectory.

The groups carry structure that was not given to the method. Of the 5800 call edges, 35.8% join two units of the same group, against 4.9% (95% range 4.2–5.8%) when the same group sizes are assigned at random ($p < 0.002$, 500 permutations): functions that call each other are embedded close to each other. Agreement with the source files is moderate (adjusted Rand index 0.36, normalised mutual information 0.63), agreement with spherical k-means with the same number of clusters is higher (ARI 0.50), and the grouping is stable under bootstrap resampling of the units (ARI 0.817, range 0.80–0.83, over 20 resamples).

Table 2: Modes of the corpus density ($m = 32$) as the concentration grows; $\kappa_N = 10.8$.

κ/κ_N	modes	singletons	largest
1/8	1	0	2326
1/4	5	0	1072
1/2	8	0	764
1	50	2	294
2	608	380	121
4	1434	1155	58
8	1904	1694	13

10.2 Calibration (E2)

On the 57 Q-doc test questions (132 582 scored pairs, 57 relevant), the kernel posterior has expected calibration error 1×10^{-4}, Brier score 4×10^{-4} and log loss 2.2×10^{-3}, against 0.010, 0.0024 and 0.013 for the logistic fallback. Platt scaling fitted on the same calibration set is as good as the kernel posterior, and the isotonic projection changed nothing measurable here, because the estimated posterior was already monotone. These summary figures are dominated by the many non-relevant pairs and say little about the region that matters; the decisive quantity is the score at which the posterior reaches τ. With prior $\pi = 1/2326$ it is $z = 6.06$ standard deviations above the question's mean score for $\tau = 0.25$ (4.48 for $\tau = 0.1$, 6.85 for $\tau = 0.5$).

10.3 Retrieval (E1)

Table 3 compares retrieval rules on the 57 Q-doc test questions. The embedder ranks the relevant function first for 23% of the questions and within the top ten for 72% (mean reciprocal rank 0.39). The calibrated trace at $\tau = 0.25$ accepts one unit for almost every question; its precision, 0.24, matches what a calibrated threshold of 0.25 promises, but its recall is only 0.25. This is the correct behaviour of the rule given the evidence: the posterior of the second-ranked unit is rarely above one in four, so the rule declines to add it. Fixed top-k reaches a recall of 0.72 at $k = 10$ by accepting nine irrelevant units for each relevant one. The split-conformal threshold delivers its guarantee (recall 0.90 at $\alpha = 0.1$) at a cost of 74 units and 16 200 tokens per question, far beyond

Table 3: Retrieval on 57 Q-doc test questions: recall of the relevant function, precision, mean set size and mean tokens. 95% intervals for recall in brackets.

rule	recall	prec.	size	tokens
top-1	0.23 [0.12,0.35]	0.23	1.0	223
top-3	0.49 [0.35,0.61]	0.16	3.0	618
top-5	0.60 [0.46,0.72]	0.12	5.0	1038
top-10	0.72 [0.60,0.83]	0.07	10.0	2056
top-20	0.79 [0.67,0.90]	0.04	20.0	4363
largest gap	0.49 [0.35,0.61]	0.26	3.3	664
PageRank top-10	0.70 [0.58,0.83]	0.07	10.0	2047
conformal α=0.2	0.84 [0.74,0.93]	0.03	38.5	8668
conformal α=0.1	0.90 [0.81,0.97]	0.01	73.7	16203
trace τ=0.5	0.23 [0.12,0.35]	0.23	1.0	223
trace τ=0.25	0.25 [0.14,0.35]	0.24	1.0	225
trace τ=0.1	0.42 [0.28,0.54]	0.21	3.4	787
without graph	0.42 [0.28,0.54]	0.21	3.3	763

the context of a small model. Graph evidence did not change the outcome measurably: with edges removed, the trace returns the same sets at $\tau = 0.25$ and nearly the same at $\tau = 0.1$. The lifts estimated from the served model were $\lambda_{\text{callee}} = 2.0$, $\lambda_{\text{caller}} = 3.5$ and $\lambda_{\text{sibling}} = 1.2$; by Proposition 3 a single callee edge can then lift only units whose own posterior is at least 0.14, a caller edge only those at 0.09 or more, and few units other than the seeds reach either.

Compressed codes. RaBitQ codes take 136 bytes per unit against 1032 for int8. Without rescoring, the top ten by RaBitQ agree with the exact top ten in 87% of positions (int8: 99.6%). With the rescoring rule of Section 7, which rescored 19 units per question (0.8% of the corpus), the trace returned exactly the sets it returns on exact scores for every question (Jaccard index 1.0) for both codes.

10.4 Context rot (E4)

E4 measures the loss of a reference answer (the documentation of the function) under the served model, in nats per token, for different contexts. It ran on the six Q-gen test questions, so its intervals are wide and its results are indicative only. Without context the loss is 5.66; with the relevant function alone it is 0.60. Distractors raise it steadily, and similar (hard) distractors more than random ones: with 2, 4, 8 and 16 hard distractors the loss is 0.82, 0.92, 1.14 and 1.21, with random ones 0.62, 0.79, 0.92 and 0.98. Top-k contexts are worse still (top-10: 1.12; top-20: 2.05), because they combine many hard distractors with the length cost reported in the literature [16, 31]. The trace context had a loss of 2.16, worse than the oracle because the trace missed the relevant function for two of the six questions. Quality pruning reduced that context from 647 to 393 tokens and raised its quality Q from 0.17 to 0.39 without a measurable change in loss (2.27; the intervals overlap widely). Across the 78 contexts that contain the relevant function, the rot index $1 - Q$ correlates positively with the increase in loss over the oracle context (Spearman $\rho = 0.36$). The direction supports the index as a measure of harmful context; the strength is modest and the sample small. Prompt evaluation time is proportional to length (Spearman $\rho = 0.99$ over 84 prompts), so every pruned token is also saved latency.

Table 4: Median scan time (ms) of 10^6 RaBitQ codes by block size and threads.

block	1 thread	8 threads	24 threads
1 KiB	16.4	20.3	160.9
4 KiB	15.7	9.0	40.6
16 KiB	15.2	7.8	7.8
64 KiB	14.4	8.0	6.1
256 KiB	15.3	7.3	17.5
1 MiB	15.5	11.8	17.3

10.5 Execution (E5)

Over 6 168 checked runs the executor recorded no violation of admission or commit order, and every result (scan scores, mean and standard deviation, batch scores, mode assignments and centres, iteration counts) hashed to one value across all block sizes from 1 KiB to 1 MiB and all thread counts from 1 to 24. The corpus index (2326 vectors, 316 KB of codes) is too small for parallelism to matter: a scan takes 120–220 μs at any setting. On a synthetic index of 10^6 vectors of dimension 64 (16 MB of codes), Table 4 shows the dependence on block size. With one thread the block size is irrelevant (14–17 ms). With 24 threads, blocks of 1–2 KiB are slow because workers queue at the admission gate (65% of worker time at 1 KiB), and blocks of 256 KiB and more are slow because 16 MB divides into too few blocks to balance the work over 24 threads; 16 to 64 KiB is the fast range, and 16 KiB lies within it. For mean shift on the corpus, 8–32 KiB blocks are fastest at 24 threads (0.71–0.78 s) and 1 MiB blocks are five times slower (4.0 s). The ordered executor costs between -26% and $+34\%$ against an unordered loop doing the same work, within the run-to-run variation at block sizes of 2 KiB and more. These results support a block size well below L2 and above the gate overhead; they do not isolate an L1 effect, and no cache counters were read.

10.6 Fine-tuning and quantisation (E6)

Training examples were built from 218 calibration and held-out functions (never from test functions), with 47 held out for the promotion test. Training the last two blocks (29.4 million of 630 million parameters) for two

epochs over 1244 windows of 256 tokens took 70 minutes on the CPU with a peak working set of 6.4 GB; training loss fell from 2.10 to 1.51 and validation loss from 1.80 to 1.49. On held-out domain examples the loss fell from 0.95 to 0.58 nats per token. On general text it rose from 2.62 to 5.66: the model forgot much of what it knew outside the corpus. The promotion test rejected the candidate on that ground (general loss worse by 3.05 against an allowed 0.02), although the domain improvement alone was highly significant. The test did what it is for; the result also shows that this form of training, on this little data, needs replay of general text or a penalty on the divergence from the base model before it can be promoted [7, 38].

Quantisation of the fine-tuned model was nearly lossless: from Q8_0 to Q4_K_M the mean KL divergence from the float model was 0.0003 to 0.0021 nats per token and the most probable token agreed in 99.6% of positions, at 676 to 491 MB. The repository importance matrix had no consistent effect at these bit widths (Q4_K_M: KL 0.0017 with, 0.0021 without; Q5_K_M: 0.0019 with, 0.0011 without).

10.7 Edits (E7)

Matching. The known-target protocol was run on 250 functions with nine perturbations of search/replace blocks and four of unit rewrites, each with the fuzzy rung off and on (6 500 cases). The first run exposed a defect: mimalloc defines some functions twice, in alternative branches of a conditional, and the lookup of a named unit silently took the first definition; 133 edits were applied at the wrong place. After the lookup was changed to refuse an ambiguous name, 20 of 6 500 cases (0.3%) were applied with a result that differs from the intended one, against 133 (2.0%) before. None of the 20 places an edit in the wrong function: eight differ only in the blanks of an empty line, and twelve are unit rewrites in which the line endings of a converted file, or lines inserted directly above the unit, are not reproduced. Search/replace blocks whose target occurs twice are now refused in all 250 cases. Without the fuzzy rung, a block with a one-letter typing error is refused in 242 of 250 cases; with it, 220 are applied correctly, 30 refused and none applied wrongly. The price of refusing ambiguous names is that 25 of 250 unperturbed unit rewrites, and 26 of 250 unperturbed search/replace blocks scoped to such a unit, are now refused.

With the served model. The 0.5B model could not use the edit format. Without the grammar none of its 91 answers parsed as an edit; with the grammar 44 parsed, but the model invented file paths or unit names, ran to the 768-token limit, and only two edits applied, neither equal to the target. The edited copy of the corpus still builds, because no wrong edit was written. Editing needs a larger model than the one that fits this machine's serving budget, or fine-tuning on the format.

11 Limitations

All results concern one C repository, a fork of mimalloc, on one machine, with one embedding model and one small generative model. The labelled questions are few (57 Q-doc test questions; six Q-gen questions for E4), each has a single relevant function, and the labels are derived from the corpus itself; recall against a single label underestimates the usefulness of neighbouring functions. The neighbourhood limit on the concentration and the edge lifts are heuristics whose constants were fixed by us. The block-size results are timings; cache behaviour was not observed directly. The fine-tuning experiment used one configuration and no protection against forgetting. The remaining edit deviations of Section 10 are known and not yet fixed.

12 Conclusion

Treating retrieval as a sequence of explicit decisions makes its behaviour predictable and its failures visible. The calibrated threshold returned sets whose precision matched the threshold, and it declined to add context it had no evidence for; with an embedder that ranks the right function first a quarter of the time, that means small contexts and low recall, and the remedy lies in the evidence, not in the rule. The density of the embeddings groups functions in a way that recovers the call graph, once its concentration is chosen for mode seeking rather than for likelihood. Context quality as defined here correlates with the harm a context does to a small model, though the evidence is preliminary. Ordered cache-sized execution gave bit-identical results at every setting, with block sizes from 16 to 64 KiB among the fastest. The promotion test stopped a fine-tuned model that had improved on the corpus and forgotten the rest. The natural next steps are a stronger local embedder, more and richer relevance labels, and fine-tuning with replay.

Use of AI tools. The implementation and the drafting of this paper were assisted by an AI coding assistant; the research direction, the system design and the claims are the author's.

References

[1] Aider project. Edit formats. aider.chat documentation, n.d. URL https://aider.chat/docs/more/edit-formats.html. Tool documentation. Accessed 2026-09-29.

[2] Gene M. Amdahl. Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities. In *AFIPS Conference Proceedings, Vol. 30 (Spring Joint Computer Conference)*, pages 483–485, Atlantic City, NJ, USA, April 1967. AFIPS Press. doi: 10.1145/1465482.1465560. URL https://doi.org/10.1145/1465482.1465560.

[3] Anastasios N. Angelopoulos and Stephen Bates. A Gentle Introduction to Conformal Prediction and Distribution-Free Uncertainty Quantification, 2021. URL https://arxiv.org/abs/2107.07511.

[4] Z. D. Bai, C. Radhakrishna Rao, and L. C. Zhao. Kernel estimators of density function of directional data. *Journal of Multivariate Analysis*, 27(1): 24–39, 1988. doi: 10.1016/0047-259X(88)90113-3. URL https://doi.org/10.1016/0047-259X(88)90113-3.

[5] Arindam Banerjee, Inderjit S. Dhillon, Joydeep Ghosh, and Suvrit Sra. Clustering on the unit hypersphere using von Mises-Fisher distributions. *Journal of Machine Learning Research*, 6:1345–1382, 2005. URL https://jmlr.org/papers/v6/banerjee05a.html.

[6] Tom Bergan, Owen Anderson, Joseph Devietti, Luis Ceze, and Dan Grossman. CoreDet: A Compiler and Runtime System for Deterministic Multithreaded Execution. In *Proceedings of the 15th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '10)*, pages 53–64, Pittsburgh, PA, USA, March 2010. ACM. doi: 10.1145/1736020.1736029. URL https://homes.cs.washington.edu/~djg/papers/asplos10-coredet.pdf.

[7] Dan Biderman, Jacob Portes, Jose Javier Gonzalez Ortiz, Mansheej Paul, Philip Greengard, Connor Jennings, Daniel King, Sam Havens, Vitaliy Chiley, Jonathan Frankle, Cody Blakeney, and John P. Cunningham. LoRA Learns Less and Forgets Less. *Transactions on Machine Learning Research*, August 2024. URL https://arxiv.org/abs/2405.09673. Featured Certification. https://openreview.net/forum?id=aloEru2qCG.

[8] Keith Bonawitz, Hubert Eichner, Wolfgang Grieskamp, Dzmitry Huba, Alex Ingerman, Vladimir Ivanov, Chloe Kiddon, Jakub Konečný, Stefano Mazzocchi, H. Brendan McMahan, Timon Van Overveldt, David Petrou, Daniel Ramage, and Jason Roselander. Towards Federated Learning at Scale: System Design. In *Proceedings of the 2nd SysML Conference*, Palo Alto, CA, USA, 2019. URL https://arxiv.org/abs/1902.01046.

[9] Peter Boncz, Marcin Zukowski, and Niels Nes. MonetDB/X100: Hyper-Pipelining Query Execution. In *Proceedings of the 2005 CIDR Conference*, 2005. URL https://www.cidrdb.org/cidr2005/papers/P19.pdf.

[10] Jaime Carbonell and Jade Goldstein. The Use of MMR, Diversity-Based Reranking for Reordering Documents and Producing Summaries. In *Proceedings of the 21st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 335–336, 1998. doi: 10.1145/290941.291025. URL https://www.cs.cmu.edu/~jgc/publication/The_Use_MMR_Diversity_Based_LTMIR_1998.pdf.

[11] Debashish Chakraborty, Eugene Yang, Daniel Khashabi, Dawn Lawrie, and Kevin Duh. Principled Context Engineering for RAG: Statistical Guarantees via Conformal Prediction. In *Advances in Information Retrieval (ECIR 2026)*, pages 537–546. Springer, 2026. doi: 10.1007/978-3-032-21300-6_45. URL https://arxiv.org/abs/2511.17908.

[12] Yizong Cheng. Mean shift, mode seeking, and clustering. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 17(8):790–799, 1995. doi: 10.1109/34.400568. URL https://doi.org/10.1109/34.400568.

[13] Dorin Comaniciu and Peter Meer. Mean shift: A robust approach toward feature space analysis. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 24(5):603–619, 2002. doi: 10.1109/34.1000236. URL https://doi.org/10.1109/34.1000236.

[14] Sanjoy Dasgupta and Anupam Gupta. An elementary proof of a theorem of Johnson and Lindenstrauss. *Random Structures & Algorithms*, 22 (1):60–65, 2003. doi: 10.1002/rsa.10073. URL https://doi.org/10.1002/rsa.10073.

[15] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient Finetuning of Quantized LLMs. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023. URL https://arxiv.org/abs/2305.14314.

[16] Yufeng Du, Minyang Tian, Srikanth Ronanki, Subendhu Rongali, Sravan Bodapati, Aram Galstyan, Azton Wells, Roy Schwartz, Eliu A. Huerta, and Hao Peng. Context Length Alone Hurts LLM Performance Despite Perfect Retrieval. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 23281–23298, 2025. doi: 10.18653/v1/2025.findings-emnlp.1264. URL https://aclanthology.org/2025.findings-emnlp.1264/.

[17] Charles Elkan. The Foundations of Cost-Sensitive Learning. In *Proceedings of the Seventeenth International Joint Conference on Artificial Intelligence (IJCAI)*, 2001. URL https://cseweb.ucsd.edu/~elkan/rescale.pdf.

[18] Güneş Erkan and Dragomir R. Radev. LexRank: Graph-based Lexical Centrality as Salience in Text Summarization. *Journal of Artificial Intelligence Research*, 22:457–479, 2004. doi: 10.1613/jair.1523. URL https://arxiv.org/abs/1109.2128.

[19] Shahul Es, Jithin James, Luis Espinosa Anke, and Steven Schockaert. RAGAs: Automated Evaluation of Retrieval Augmented Generation. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*, pages 150–158, 2024. doi: 10.18653/v1/2024.eacl-demo.16. URL https://aclanthology.org/2024.eacl-demo.16/.

[20] Kawin Ethayarajh. How contextual are contextualized word representations? Comparing the geometry of BERT, ELMo, and GPT-2 embeddings. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 55–65. Association for Computational Linguistics, 2019. doi: 10.18653/v1/D19-1006. URL https://aclanthology.org/D19-1006/.

[21] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. In *International Conference on Learning Representations (ICLR)*, 2023. URL https://arxiv.org/abs/2210.17323.

[22] Matteo Frigo, Charles E. Leiserson, Harald Prokop, and Sridhar Ramachandran. Cache-Oblivious Algorithms. In *40th Annual Symposium on Foundations of Computer Science (FOCS '99)*, pages 285–297, New York, NY, USA, October 1999. IEEE. doi: 10.1109/SFFCS.1999.814600. URL http://www.fftw.org/~athena/abstracts/abstract12.html.

[23] Keinosuke Fukunaga and Larry D. Hostetler. The estimation of the gradient of a density function, with applications in pattern recognition. *IEEE Transactions on Information Theory*, 21(1):32–40, 1975. doi: 10.1109/TIT.1975.1055330. URL https://doi.org/10.1109/TIT.1975.1055330.

[24] Jianyang Gao and Cheng Long. RaBitQ: Quantizing high-dimensional vectors with a theoretical error bound for approximate nearest neighbor search. *Proceedings of the ACM on Management of Data*, 2(3), 2024. doi: 10.1145/3654970. URL https://arxiv.org/abs/2405.12497.

[25] Eduardo García-Portugués. Exact risk improvement of bandwidth selectors for kernel density estimation with directional data. *Electronic Journal of Statistics*, 7:1655–1685, 2013. doi: 10.1214/13-EJS821. URL https://arxiv.org/abs/1306.0517.

[26] ggml-org/llama.cpp contributors. llama.cpp examples/training/README.md. GitHub repository ggml-org/llama.cpp, 2026. URL https://github.com/ggml-org/llama.cpp/tree/master/examples/training. Retrieved 2026-09-29.

[27] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On Calibration of Modern Neural Networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017. URL https://arxiv.org/abs/1706.04599.

[28] Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models. In *Advances in Neural Information Processing Systems 37*, 2024. doi: 10.52202/079017-1902. URL https://arxiv.org/abs/2405.14831.

[29] Peter Hall, G. S. Watson, and Javier Cabrera. Kernel density estimation with spherical data. *Biometrika*, 74(4):751–762, 1987. doi: 10.1093/biomet/74.4.751. URL https://doi.org/10.1093/biomet/74.4.751.

[30] Mark D. Hill and Michael R. Marty. Amdahl's Law in the Multicore Era. *Computer*, 41(7):33–38, July 2008. doi: 10.1109/MC.2008.209. URL https://research.cs.wisc.edu/multifacet/papers/ieeecomputer08_amdahl_multicore.pdf.

[31] Kelly Hong, Anton Troynikov, and Jeff Huber. Context Rot: How Increasing Input Tokens Impacts LLM Performance. Technical report, Chroma, jul 2025. URL https://www.trychroma.com/research/context-rot.

[32] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations (ICLR)*, 2022. URL https://arxiv.org/abs/2106.09685.

[33] Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, Kai Dang, Yang Fan, Yichang Zhang, An Yang, Rui Men, Fei Huang, Bo Zheng, Yibo Miao, Shanghaoran Quan, Yunlong Feng, Xingzhang Ren, Xuancheng Ren, Jingren Zhou, and Junyang Lin. Qwen2.5-Coder Tech-

nical Report, 2024. URL https://arxiv.org/abs/2409.12186. arXiv preprint arXiv:2409.12186.

[34] Intel Corporation. Optimizing Software for Intel performance hybrid architecture. White Paper 348851-001US, Revision 1.0, Intel Corporation, October 2021. URL https://cdrdv2-public.intel.com/685869/optimizing%20_software_for_performance_hybrid_architecture.pdf.

[35] *Intel 64 and IA-32 Architectures Optimization Reference Manual, Volume 1.* Intel Corporation, revision 049 edition, January 2024. URL https://cdrdv2-public.intel.com/814198/248966-Optimization-Reference-Manual-V1-049.pdf. Contains the Gracemont chapter (Section 4.1). Accessed 2026-09-29.

[36] Iwan Kawrakow. k-quants (llama.cpp pull request #1684). GitHub pull request #1684, ggml-org/llama.cpp, June 2023. URL https://github.com/ggml-org/llama.cpp/pull/1684. Retrieved 2026-09-29.

[37] Iwan Kawrakow. Importance Matrix calculation (llama.cpp pull request #4861). GitHub pull request #4861, ggml-org/llama.cpp, January 2024. URL https://github.com/ggml-org/llama.cpp/pull/4861. Retrieved 2026-09-29.

[38] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharshan Kumaran, and Raia Hadsell. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017. doi: 10.1073/pnas.1611835114. URL https://arxiv.org/abs/1612.00796.

[39] Monica S. Lam, Edward E. Rothberg, and Michael E. Wolf. The Cache Performance and Optimizations of Blocked Algorithms. In *Proceedings of the Fourth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS IV)*, pages 63–74, Palo Alto, CA, USA, 1991. ACM. doi: 10.1145/106972.106981. URL https://doi.org/10.1145/106972.106981.

[40] Viktor Leis, Peter Boncz, Alfons Kemper, and Thomas Neumann. Morsel-Driven Parallelism: A NUMA-Aware Query Evaluation Framework for the Many-Core Age. In *Proceedings of the 2014 ACM SIGMOD International Conference on Management of Data*, pages 743–754, Snowbird, UT, USA, June 2014. ACM. doi: 10.1145/2588555.2610507. URL https://db.in.tum.de/~leis/papers/morsels.pdf.

[41] Hui Lin and Jeff Bilmes. A Class of Submodular Functions for Document Summarization. In *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies*, pages 510–520, 2011. URL https://aclanthology.org/P11-1052/.

[42] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration. In *Proceedings of the 7th MLSys Conference (MLSys)*, 2024. URL https://arxiv.org/abs/2306.00978.

[43] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024. doi: 10.1162/tacl_a_00638. URL https://arxiv.org/abs/2307.03172.

[44] Tongping Liu, Charlie Curtsinger, and Emery D. Berger. Dthreads: Efficient Deterministic Multithreading. In *Proceedings of the 23rd ACM Symposium on Operating Systems Principles (SOSP '11)*, pages 327–336. ACM, October 2011. doi: 10.1145/2043556.2043587. URL https://people.cs.umass.edu/~emery/pubs/dthreads-draft.pdf.

[45] John M. Mellor-Crummey and Michael L. Scott. Algorithms for Scalable Synchronization on Shared-Memory Multiprocessors. *ACM Transactions on Computer Systems*, 9(1):21–65, February 1991. doi: 10.1145/103727.103729. URL https://www.cs.rochester.edu/u/scott/papers/1991_TOCS_synch.pdf.

[46] Microsoft. mimalloc: a compact general purpose allocator. https://github.com/microsoft/mimalloc, 2026. Commit 31d034d9, version 3.5.3. The Demo repository is a fork of this project.

[47] Ali Modarressi, Hanieh Deilamsalehy, Franck Dernoncourt, Trung Bui, Ryan A. Rossi, Seunghyun Yoon, and Hinrich Schütze. NoLiMa: Long-Context Evaluation Beyond Literal Matching. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025. URL https://arxiv.org/abs/2502.05167.

[48] Marek Olszewski, Jason Ansel, and Saman Amarasinghe. Kendo: Efficient Deterministic Multithreading in Software. In *Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '09)*, pages 97–108, Washington, DC, USA, March 2009. ACM. doi: 10.1145/1508244.

1508256. URL https://commit.csail.mit.edu/papers/2009/asplos073-olszewski.pdf.

[49] Emanuel Parzen. On estimation of a probability density function and mode. *The Annals of Mathematical Statistics*, 33(3):1065–1076, 1962. doi: 10.1214/aoms/1177704472. URL https://doi.org/10.1214/aoms/1177704472.

[50] Thanumalayan Sankaranarayana Pillai, Vijay Chidambaram, Ramnatthan Alagappan, Samer Al-Kiswany, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. All file systems are not created equal: On the complexity of crafting crash-consistent applications. In *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI 14)*, pages 433–448. USENIX Association, 2014. URL https://www.usenix.org/conference/osdi14/technical-sessions/presentation/pillai.

[51] Miloš Radovanović, Alexandros Nanopoulos, and Mirjana Ivanović. Hubs in space: Popular nearest neighbors in high-dimensional data. *Journal of Machine Learning Research*, 11(86):2487–2531, 2010. URL https://jmlr.org/papers/v11/radovanovic10a.html.

[52] Murray Rosenblatt. Remarks on some nonparametric estimates of a density function. *The Annals of Mathematical Statistics*, 27(3):832–837, 1956. doi: 10.1214/aoms/1177728190. URL https://doi.org/10.1214/aoms/1177728190.

[53] Yuling Shi, Yichun Qian, Hongyu Zhang, Beijun Shen, and Xiaodong Gu. LongCodeZip: Compress Long Context for Code Language Models. In *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 141–153, 2025. doi: 10.1109/ASE63991.2025.00020. URL https://arxiv.org/abs/2510.00446.

[54] Charles J. Stone. Optimal rates of convergence for nonparametric estimators. *The Annals of Statistics*, 8(6):1348–1360, 1980. doi: 10.1214/aos/1176345206. URL https://doi.org/10.1214/aos/1176345206.

[55] Tree-sitter project. Tree-sitter: Introduction. tree-sitter.github.io documentation, n.d. URL https://tree-sitter.github.io/tree-sitter/. Project documentation. Accessed 2026-09-29.

[56] Jingjing Wang, Xiwen Chen, Wenhui Zhu, Huayu Li, Zhengxiao He, Feiyang Cai, Ana S. Carreon-Rascon, Xuanzhao Dong, and Feng Luo. Context Pruning for Coding Agents via Multi-Rubric Latent Reasoning, 2026. URL https://arxiv.org/abs/2605.15315. arXiv comment: COLM 2026 Workshop on Efficient Reasoning.

[57] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *38th Conference on Neural Information Processing Systems (NeurIPS)*, 2024. URL https://arxiv.org/abs/2405.15793. arXiv:2405.15793.

[58] Tianjun Zhang, Shishir G. Patil, Naman Jain, Sheng Shen, Matei Zaharia, Ion Stoica, and Joseph E. Gonzalez. RAFT: Adapting Language Model to Domain Specific RAG, 2024. URL https://arxiv.org/abs/2403.10131. arXiv preprint arXiv:2403.10131.

[59] Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, Fei Huang, and Jingren Zhou. Qwen3 Embedding: Advancing Text Embedding and Reranking Through Foundation Models, 2025. URL https://arxiv.org/abs/2506.05176. arXiv:2506.05176.

[60] Yikun Zhang and Yen-Chi Chen. Kernel smoothing, mean shift, and their learning theory with directional data. *Journal of Machine Learning Research*, 22(154):1–92, 2021. URL https://jmlr.org/papers/v22/20-1194.html.

[61] Yilin Zhang, Xinran Zhao, Zora Zhiruo Wang, Chenyang Yang, Jiayi Wei, and Tongshuang Wu. cAST: Enhancing Code Retrieval-Augmented Generation with Structural Chunking via Abstract Syntax Tree. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 8106–8116, 2025. URL https://aclanthology.org/2025.findings-emnlp.430/.